Issue key	Summary	Description
NY4-57	Create GET Model Inference	User Story: * As a job seeker, I want the platform to generate job recommendations based on my resume's content, so that I can see how well I match open roles. Acceptance Criteria: * A GET endpoint /model-inference accepts a resumeID as part of a body. * The resume is fetched and passed to the ML model for inference. * The response contains a list of job IDs and their respective match scores. * Model inference is completed and returned. Benefit/Value: * Makes the recommendation process feel intelligent and dynamic, helping users feel seen and understood by the system.
NY4-56	Create POST Process Resume	User Story: * As a job seeker, I want the app to process my uploaded resume and return job matches, so that I can start browsing job options personalized to my background. Acceptance Criteria: * A POST endpoint /process-resume accepts file uploads via multipart/form-data. * Resume content is extracted and passed to the ML model. * The endpoint returns a JSON response containing a resumeID and a list of matched jobs. * Validation is included for file format and size. * Errors return appropriate status codes with user-friendly messages. Benefit/Value: * Allows users to initiate their personalized job search instantly through a simple upload, reducing effort and manual input.
NY4-55	Call Backend from Frontend UI	User Story: * As a job seeker, I want my resume to be analyzed as soon as I upload it, so that I can begin swiping through job matches without delay. Acceptance Criteria: * The frontend calls the Flask backend /process-resume endpoint when a resume file is selected. * On success, a resumeID is saved to local storage for session tracking. * If successful, the user can navigate to the /search page to begin swiping. Benefit/Value: * Seamless integration ensures users can quickly move from upload to results, making the app feel fast and intuitive.

NY4-54	Backend Query Job Dataset	User Story: * As a job seeker, I want the platform to match me with real job postings based on my resume, so that the recommendations I receive are relevant and up-to-date. Acceptance Criteria: * Backend connects to a structured dataset (CSV, database, or API) of job postings. * A function is implemented to retrieve relevant jobs based on skills, keywords, or embedding similarity. * Only verified or curated jobs are returned. * At least 20 jobs are returned per resume request. * Returned job objects include id, title, description, and metadata used by the frontend. Benefit/Value: * Provides users with trusted, personalized job options that feel meaningful, accurate, and current.
NY4-49	Deploy ML model to Sagemaker	User Story: * As a job seeker, I want accurate and fast job recommendations based on my resume, so that I can quickly discover relevant job opportunities. Acceptance Criteria: * The trained ML model is deployed as a SageMaker endpoint that accepts resume input and returns job recommendations. * The endpoint is accessible via a secure HTTP API. * The model returns predictions within 1 second for a typical resume file. * Model output includes a list of job IDs, match percentages, and any additional metadata used by the frontend. * Model uptime is monitored, and errors are logged or handled gracefully. Benefit/Value: * Ensures the intelligence of the platform is scalable and fast, allowing users to receive personalized recommendations without delays or errors.
NY4-48	Cross browser testing	As a user, I should be able to visit and use the website from any browser without errors. AC: Ensure that ResuMInd works and looks correct across different common web browsers

NY4-47	Implement Form Inputs	User Story: * As a new user, I want to enter my name and upload my resume easily, so that I can quickly get started and receive job suggestions personalized to me. Acceptance Criteria: * The homepage includes a text input for name and a resume upload button. * The resume input accepts .pdf, .doc, and .docx files. * Upon selection, the file is stored and ready to be sent to the backend. * The name input can be optionally used to personalize UI text. Benefit/Value: * Smooth onboarding helps users feel confident and engaged right away, minimizing friction in beginning the job search process.
NY4-33	Create props for each part of the job application card	User Story: * As a job seeker, I want each job card to clearly display relevant job details (title, description, match score, etc.), so that I can make informed decisions while swiping. Acceptance Criteria: * The job card component accepts props for title, description, matchScore, and likelihood. * Props are dynamically populated from job data received from the backend. * Cards render correctly with various prop values and show placeholder text if a prop is missing. * The component is reusable across the swipe and matches pages. * A test card renders correctly with mock data during development. Benefit/Value: * Enables a scalable and flexible UI for showing personalized job information, helping users evaluate roles at a glance.
NY4-30	Create backend API using Flask	User Story: * As a job seeker, I want the app to process my uploaded resume and return job matches, so that I can get personalized recommendations without needing technical knowledge. Acceptance Criteria: * Flask API exposes an endpoint /process-resume that accepts file uploads via POST. * The uploaded resume is forwarded to the ML model and the results are returned in JSON format. * API response includes a resumeID and list of matched jobs. * CORS is enabled so the frontend can make cross-origin requests. * Returns appropriate status codes for success (200), client errors (400), or server errors (500). Benefit/Value: * This bridges the frontend and ML model, allowing users to submit resumes and receive recommendations without ever leaving the app.

NY4-29	Research Deployment Methods	User Story: * As a developer working on ResuMind, I want to identify the best deployment strategy for the backend and ML model, so that users experience fast and reliable service. Acceptance Criteria: * At least 2 deployment methods are evaluated (e.g., AWS SageMaker, Lambda, EC2) * Each method is assessed for cost and integration with Flask. * Recommendations are documented clearly in a markdown or PDF report. * Team agrees on a method based on findings for MVP launch. Benefit/Value: * Ensures long-term stability and maintainability of the platform, improving the user experience and enabling smooth growth.
NY4-19	Create test cases sheet	User Story: * As a developer, I want a centralized test case sheet that outlines how each feature will be tested, so that I can systematically verify functionality and ensure the app works as expected. Acceptance Criteria: * A test case sheet is created and shared with the team. * Test cases are written for the currently implemented user stories. * The sheet is stored in a shared location (shared drive). * The team has access and can contribute additional test cases as features are developed. Benefit/Value: * Provides a structured way to verify that user stories meet their acceptance criteria, reducing bugs and improving team confidence in the quality of the application